

Архивация данных — это процесс упаковки и сжатия одного или нескольких файлов в один специальный файл (архив) для экономии места на диске и удобства их передачи

Главные задачи архивации

- **Экономия памяти:** Специальные алгоритмы находят повторения в коде файлов и сокращают их размер без потери информации.
- **Объединение данных:** Позволяет превратить тысячи мелких файлов и папок в один документ, что сильно ускоряет их копирование.
- **Удобство передачи:** Один сжатый файл гораздо проще отправить по электронной почте, через мессенджеры или загрузить в облако.
- **Защита информации:** Большинство архиваторов позволяют установить пароль и зашифровать содержимое от посторонних.

Виды сжатия данных делятся на две основные категории: сжатие без потерь (Lossless) и сжатие с потерями (Lossy)

Сжатие без потерь (Lossless)

При этом методе данные сжимаются так, что при распаковке получается **абсолютно точная копия** оригинала. Каждый байт возвращается на свое место.

- **Где применяется:** Архивация документов (.docx, .pdf), баз данных, программного кода (.exe, .py), системных файлов и таблиц. Потеря даже одного символа здесь разрушит файл.
- **Как это работает:** Алгоритмы ищут закономерности и повторы. Например, вместо записи «синий, синий, синий, синий» алгоритм запишет «синий x4».
- **Основные форматы:**
 - **Для файлов:** .zip, .7z, .rar, .tar.gz
 - **Для аудио:** .flac, .alac
 - **Для изображений:** .png, .bmp, .tiff

2. Сжатие с потерями (Lossy)

Этот метод безвозвратно удаляет часть информации, которую человеческое ухо или глаз почти не замечают. Это позволяет уменьшить размер файла в десятки раз.

- **Где применяется:** Мультимедиа (фото, видео, музыка, стриминг), где важнее скорость передачи и экономия места, а не идеальная точность.

- **Как это работает:** Используются особенности человеческого восприятия. Например, удаляются слишком высокие звуки, которые человек не слышит, или усредняются похожие оттенки пикселей на картинке.
- **Основные форматы:**
 - **Для изображений:** .jpeg, .webp
 - **Для аудио:** .mp3, .aac, .ogg
 - **Для видео:** .mp4 (кодеки H.264, H.265/HEVC), .mkv

Характеристик	Сжатие без потерь	Сжатие с потерями
Точность копии	100% совпадение с оригиналом	Часть данных удаляется навсегда
Размер файла	Уменьшается в 1.5 – 3	Уменьшается в 10 – 50+
Качество	Идеальное (оригинальное)	Снижается при сильном
Повторное сжатие	Безопасно, качество не теряется	Каждый раз качество ухудшается

Сжатие с потерями (Lossy compression) — это метод уменьшения размера файлов путем безвозвратного удаления избыточной информации, которую человеческие органы чувств почти не воспринимают.

Этот метод жертвует математической точностью ради экстремальной экономии места. Файл может уменьшиться в 10, 20 или даже 100 раз, что невозможно при сжатии без потерь.

Как это работает: Психофизиология на службе ИТ

Алгоритмы сжатия с потерями обманывают наше восприятие, используя несовершенство человеческого зрения и слуха.

1. В звуке (Акустическое маскирование)

- **Удаление неслышимого:** Из аудиофайла полностью вырезаются частоты выше 20 кГц, которые человеческое ухо физически не способно уловить.

- **Эффект маскировки:** Если одновременно звучат громкий и тихий звуки (например, удар барабана и шелест струны), мозг слышит только громкий. Алгоритм удаляет тихий звук, так как для слушателя он все равно скрыт.
- *Пример:* Так работает формат **MP3** или **AAC**.

2. В изображениях (Цветовая субдискретизация)

- **Приоритет яркости:** Человеческий глаз гораздо чувствительнее к перепадам яркости (светлое/темное), чем к изменениям цвета.
- **Усреднение пикселей:** Алгоритм делит картинку на блоки (обычно 8x8 пикселей). Он сохраняет точную карту яркости, но усредняет цвета похожих соседних пикселей. Вместо 10 близких оттенков синего на небе записывается один средний.
- *Пример:* Так работает формат **JPEG**.

3. В видео (Межкадровое сжатие)

- **Статичный фон:** Видео — это поток картинок (кадров). Если в сцене человек говорит на фоне стены, сама стена не меняется.
- **Запись изменений:** Вместо того чтобы сохранять стену заново в каждом кадре, алгоритм сохраняет ее один раз, а в следующих кадрах записывает только мимику лица движения человека.
- *Пример:* Так работают кодеки **H.264** и **AV1**.

Главные правила и ограничения

- **Эффект «копирки» (Поколенческая потеря качества):** Если вы откроете JPEG-картинку, отредактируете и снова сохраните в JPEG, она сожмется второй раз. Часть данных удалится снова. Если повторить это 10 раз,

изображение покроется «кубиками» (артефактами) и станет непригодным.

- **Битрейт решает все:** Качество сжатия настраивается. В MP3 битрейт 320 kbps дает отличный звук, а 96 kbps превратит музыку в «телефонный» гул.
- **Точка невозврата:** Восстановить удаленные данные невозможно. Нельзя из MP3 файла сделать обратно полноценный CD-диск (FLAC). Звуки физически стерты из файла.

Где без этого сжатия не обойтись

Без сжатия с потерями современный интернет перестал бы работать из-за перегрузки каналов:

- **Стриминг (YouTube, Netflix, Twitch):** Без сжатия 4K-видео требовало бы скорость интернета в несколько гигабит в секунду.
- **Онлайн-музыка (Spotify, Яндекс Музыка):** Позволяет слушать треки мгновенно даже при плохом мобильном интернете.
- **Социальные сети:** Смартфоны делают фото весом в 5–10 МБ, но VK, Telegram и WhatsApp автоматически сжимают их до 200–500 КБ для мгновенной отправки.

Сжатие без потерь (Lossless compression) — это метод уменьшения размера файлов, который гарантирует абсолютное, побитовое совпадение распакованных данных с оригиналом.

В отличие от сжатия с потерями, здесь не удаляется ни один байт информации. Процесс полностью обратим: из сжатого архива всегда можно восстановить исходный файл в первозданном виде [ру].

Как это работает: Магия алгоритмов

Сжатие без потерь основано на устранении **статистической избыточности**. Алгоритмы ищут в файле закономерности, повторы или предсказуемые последовательности и записывают их более компактным кодом.

Вот три классических алгоритма, на которых строится большинство архиваторов:

1. RLE (Run-Length Encoding — Кодирование длин серий)

Самый простой алгоритм. Он заменяет цепочки одинаковых символов на один символ и число его повторений.

- *Оригинал (20 символов):* ААААААААААББББВВВВВВГГ
- *После сжатия (8 символов):* А9Б4В5Г2
- *Эффективность:* Отлично сжимает простые рисунки (например, чертежи или иконки, где много пикселей одного цвета подряд), но бесполезен для обычного текста.

2. Алгоритм Хаффмана (Кодирование по частоте)

Алгоритм анализирует, какие символы встречаются часто, а какие — редко. Частым символам он присваивает очень короткие двоичные коды (например, 2 бита), а редким — длинные (например, 16 бит).

- *Пример:* В русском тексте буквы «О» и «А» встречаются постоянно, а «Ф» и «Ъ» — редко. Вместо того чтобы тратить на каждую букву стандартные 8 или 16 бит, алгоритм Хаффмана «ужимает» самые ходовые буквы, за счет чего уменьшается общий размер текста.

3. LZ77 и LZ78 (Словарное сжатие)

Этот алгоритм лежит в основе форматов **ZIP** и **GZIP**. Он ищет в тексте повторяющиеся слова или целые фразы и заменяет их текстовыми ссылками на уже встреченный фрагмент (словарное совпадение).

- *Пример:* Если в договоре 50 раз встречается фраза «Общество с ограниченной ответственностью», алгоритм запишет её полностью только один раз. В остальных 49 случаях он поставит короткую ссылку вида (*вернуться назад на 400 символов, скопировать 40 символов*).

Главные особенности и ограничения

- **Ограниченный коэффициент сжатия:** Данные редко удастся сжать более чем в 2–4 раза. Ужать текстовый документ или базу данных в 50 раз без потери информации физически невозможно.
- **Зависимость от структуры:** Метод работает только там, где есть избыточность. Если файл уже состоит из хаотичных данных без повторов, алгоритму не за что «зацепиться».
- **Парадокс «сжатия сжатого»:** Если попытаться заархивировать файл, который уже был зажат (например, фильм **.mp4** или музыку **.mp3**), то размер архива не уменьшится, а может даже стать чуть больше оригинала, так как архиватору придется добавить в файл свои служебные заголовки.

Где применяется сжатие без потерь

Этот вид сжатия используется везде, где критически важна точность и порча хотя бы одного бита приведет к поломке файла:

1. **Документы и код:** Тексты (`.txt`, `.docx`), таблицы (`.xlsx`), базы данных (`.sql`), исполняемые файлы программ (`.exe`, исполняемые файлы в Linux) `[py]`. Применяются форматы `.zip`, `.7z`, `.rar`, `.tar.gz`.
2. **Профессиональная фотография и графика:** Форматы `.png` (для графики и интерфейсов), `.tiff` и `.raw` (для сохранения максимума деталей при съемке).
3. **Высококачественный звук:** Форматы `.flac` и `.alac` (Apple Lossless). Они весят в 2–3 раза больше, чем MP3, но сохраняют студийное звучание аудиозаписи.
4. **Медицинские и научные данные:** Снимки МРТ, результаты космических съемок, сейсмические данные. Ошибки здесь могут стоить человеческих жизней или сорвать научные исследования.

Если вам интересно, как эти алгоритмы работают на практике, мы можем разобрать, **какой формат (ZIP, 7z или RAR) сжимает файлы лучше всего** и в чем их техническая разница. Что думаете?

История алгоритмов сжатия данных — это эволюция от простых допущений до сложнейших математических моделей, которая развивалась параллельно с дефицитом цифровой памяти. [1, 2]

Развитие технологий сжатия можно разделить на пять ключевых исторических эпох. [2, 3]

Эпоха 0: Предыстория (XIX век)

Концепция экономии места при передаче информации появилась задолго до первых компьютеров. [4]

- **1838 год — Код Морзе:** Самый первый пример сжатия на основе частотного анализа. Изобретатель Сэмюэл Морзе заметил, что в английском языке буквы «Е» и «Т» встречаются чаще всего, и присвоил им кратчайшие сигналы (точка и тире), оставив редким буквам (например, «Q» или «Z») длинные комбинации. [2, 4, 5]

Эпоха 1: Рождение теории информации (1940-1950-е)

Период формирования фундаментальной математической базы. [2]

- **1948 год — Теория Клода Шеннона:** Отец цифровых коммуникаций доказал, что у любого текста или файла есть предел сжатия (информационная энтропия), выше которого ужать данные без потерь невозможно. [6, 7]
- **1949 год — Алгоритм Шеннона — Фано:** Первая попытка эффективного побитового кодирования символов. [4]
- **1952 год — Алгоритм Хаффмана:** Студент Дэвид Хаффман превзошел своих преподавателей и создал математически безупречный алгоритм частотного кодирования с помощью бинарных деревьев. Его метод до сих пор используется как финальный этап в большинстве современных архиваторов. [3, 6]

Эпоха 2: Словарная революция (1970-1980-е)

С появлением первых дискет и онлайн-хранилищ возникла острая потребность сжимать не просто отдельные буквы, а целые слова. [5, 8]

- **1977-1978 годы — Алгоритмы LZ77 и LZ78:** Израильские ученые Абрахам Лемпель и Яков Зив совершили революцию. Вместо подсчета частоты букв они предложили использовать динамический «словарь» (скользящее окно), заменяя повторяющиеся фразы ссылками на их прошлые упоминания. [4, 8]
- **1984 год — Алгоритм LZW:** Терри Велч модифицировал LZ78. Этот алгоритм лег в основу формата изображений **GIF** и утилиты сжатия в UNIX. [2, 3]
- **Конец 1980-х — Рождение ZIP:** Фил Кац создает алгоритм **DEFLATE** (комбинация LZ77 и Хаффмана) и выпускает программу PKZIP. Этот формат до сих пор остается самым совместимым в мире. [3]

Эпоха 3: Мультимедийный взрыв и сжатие с потерями (1990-е)

Компьютеры научились работать со звуком и графикой, но несжатые фотографии и песни весили слишком много для жестких дисков того времени. [2, 9]

- **1992 год — Стандарт JPEG:** Разработан метод сжатия картинок на основе Дискретного косинусного преобразования (DCT). [2, 10]
- **1993 год — Формат MP3 (MPEG-1 Audio Layer III):** Ученые из общества Фраунгофера научили алгоритмы выбрасывать из аудиофайлов звуки, которые не слышит человеческое ухо. Треки уменьшились в 10–12 раз, породив эпоху цифрового пиратства и плееров iPod.
- **1990-е — Продвинутое архиваторы:** Евгений Рошал создает формат **RAR** (1993), а Игорь Павлов — бесплатный алгоритм **LZMA** и архиватор **7-Zip** (1999), который до сих пор удерживает лидерство по плотности сжатия файлов без потерь. [3]

Эпоха 4: Современность, Big Data и нейросети (2010-2020-е)

Главным приоритетом стала не максимальная плотность сжатия, а **скорость обработки петабайтов данных** на интернет-серверах.

- **Алгоритмы от ИТ-гигантов:** Google создает алгоритмы **Brotli** (для ускорения загрузки веб-страниц) и **Zopfli**. Facebook (ныне Meta) выпускает **Zstandard (zstd)** — невероятно быстрый алгоритм, который мгновенно сжимает базы данных «на лету».
- **Нейросети и ИИ:** Современные видеокодеки (например, **AV1** или H.266) используют искусственный интеллект для генерации и «дорисовывания» недостающих пикселей при трансляции потокового видео в 4K и 8K.

Если вам интересно, мы можем детальнее разобрать **алгоритм Хаффмана** или **LZ77**, чтобы понять, как устроена их логика изнутри. Какой из них вас привлекает больше?

Сжатие файлов в Linux выполняется либо через графический интерфейс (GUI), либо через командную строку (Терминал).

В Linux процессы объединения файлов в один пакет (архивация) и уменьшения их размера (сжатие) разделены, поэтому большинство архивов имеют двойное расширение, например `.tar.gz` или `.tar.xz`.

Способ 1: Сжатие через Терминал (CLI)

Это самый мощный способ. Откройте терминал (`Ctrl + Alt + T`) и используйте одну из следующих команд в зависимости от нужного формата.

1. Формат TAR.GZ (Самый популярный, оптимальный баланс скорости и размера)

Использует утилиту `tar` совместно со сжатием `gzip`.

- Команда для сжатия папки или файлов: `tar -czvf имя_архива.tar.gz /путь/к/папке_или_файлу`
- *(Ключи: `c` — создать архив, `z` — сжать через `gzip`, `v` — показывать процесс, `f` — записать в файл).*
- Команда для распаковки: `tar -xzvf имя_архива.tar.gz`
-

2. Формат TAR.XZ (Максимальное сжатие для долгосрочного хранения)

Сжимает файлы сильнее, чем `gzip`, но требует чуть больше времени и ресурсов процессора.

- Команда для сжатия: `tar -cjvf имя_архива.tar.xz /путь/к/папке_или_файлу`
-
- Команда для распаковки: `tar -xjvf имя_архива.tar.xz`
-

3. Классический формат ZIP (Для совместимости с Windows)

Идеально, если архив нужно будет открывать на компьютерах с Windows без установки сторонних программ.

- Команда для сжатия папки со всем содержимым: `zip -r имя_архива.zip /путь/к/папке`
- *(Ключ `-r` означает «рекурсивно», то есть сжимать вместе со всеми подпапками и файлами внутри).*
- Команда для распаковки: `unzip имя_архива.zip`
-

Способ 2: Сжатие через графический интерфейс (GUI)

Если вы работаете в пользовательских дистрибутивах (Ubuntu, Linux Mint, Fedora, Manjaro), интерфейс встроенного проводника позволяет сжимать файлы в пару кликов.

1. Откройте файловый менеджер и **выделите** нужные файлы или папки.
2. Нажмите по ним **правой кнопкой мыши**.
3. В контекстном меню выберите пункт «Сжать...» (в некоторых графических окружениях он называется «Создать архив...»).
4. В появившемся окне введите имя архива и выберите желаемый формат из выпадающего списка (.zip, .tar.gz, .7z).
5. Нажмите кнопку «Создать».

Полезный лайфхак: Как сжать файл «на лету» без создания TAR

Если вам нужно сжать один конкретный файл (например, огромный лог-текст `server.log`), не нужно объединять его в tar-пакет. Можно использовать одиночные утилиты:

- `gzip server.log` — мгновенно сожмет файл и превратит его в `server.log.gz` (исходный файл исчезнет).
- `gunzip server.log.gz` — вернет файл в исходное состояние.

Вам требуется автоматизировать этот процесс (например, написать **скрипт бэкапа**) или вы просто подбираете команду для разового использования?

Сжатие файлов в Windows выполняется либо встроенными средствами системы, либо с помощью сторонних бесплатных программ.

Выбор метода зависит от того, нужно ли вам просто упаковать файлы для отправки или максимально уменьшить их размер.

Способ 1: Встроенные средства Windows (Формат ZIP)

Этот способ работает «из коробки» на любой версии Windows (7, 10, 11). Созданный архив гарантированно откроется на любом другом компьютере или смартфоне.

1. **Выделите** один или несколько файлов/папок, которые нужно сжать.
2. Нажмите по ним **правой кнопкой мыши**.
3. Вызовите команду сжатия:
 - **В Windows 11:** В появившемся меню выберите «Сжать в ZIP-файл».
 - **В Windows 10 и старых версиях:** Выберите пункт «Отправить» $\rightarrow$ «Сжатая ZIP-папка».
- 4.
5. Рядом появится архив. При желании введите для него новое имя и нажмите **Enter**.

Для извлечения файлов: Кликните по ZIP-архиву правой кнопкой мыши и выберите «Извлечь все...».

Способ 2: С помощью бесплатной программы 7-Zip (Формат 7z)

Встроенный ZIP-архиватор сжимает данные слабо. Если вам нужно ужать файлы **максимально сильно** или поставить на архив **пароль**, используйте бесплатную утилиту **7-Zip** (скачать её можно с официального сайта 7-zip.org).

1. **Выделите** нужные файлы и нажмите **правой кнопкой мыши**.
2. В контекстном меню выберите:
 - **В Windows 11:** Нажмите «Показать дополнительные параметры» → **«7-Zip»** → «Добавить к архиву...».
 - **В Windows 10:** Сразу выберите **«7-Zip»** → «Добавить к архиву...».
- 3.
4. В открывшемся окне настроек вы можете:
 - Изменить формат архива на **7z** (он обеспечивает самое сильное сжатие).
 - Выбрать уровень сжатия (например, «Ультра» для максимального эффекта).
 - В блоке «Шифрование» ввести **пароль**, чтобы никто не смог открыть ваши файлы без разрешения.
- 5.
6. Нажмите «ОК».

Способ 3: Через командную строку (Для продвинутых пользователей)

Если вы хотите автоматизировать процесс сжатия с помощью скриптов (**.bat** файлов), в Windows 10 и 11 есть встроенная консольная утилита **tar** (аналог Linux-версии).

1. Откройте командную строку (нажмите **Win + R**, введите **cmd** и нажмите **Enter**).
2. Введите команду для создания ZIP-архива: **tar -a -cf имя_архива.zip "путь_к_папке_или_файлу"**
3. *(Ключ **-a** автоматически определяет формат сжатия по расширению файла, а **-cf** создает новый архив).*

Подскажите, файлы какого **типа и объема** вы планируете сжимать (например, текстовые документы на 10 МБ или видео/фото на 50 ГБ)? Я помогу определить, какой способ даст **наилучший результат**.

Архивация файлов на macOS выполняется встроенными инструментами через графический интерфейс Finder, утилиту Терминал или сторонние программы. Штатным форматом сжатия в macOS является **ZIP**. [1]

Способ 1: Сжатие через Finder (Графический интерфейс)

Самый быстрый способ, не требующий установки стороннего софта. [2]

1. Откройте **Finder** и найдите файлы или папки, которые нужно сжать.
 2. Выделите их и нажмите правой кнопкой мыши (или удерживайте клавишу **Control** при клике).
 3. В контекстном меню выберите пункт «Сжать» (после этого слова будет указано имя папки или количество файлов). [1, 3]
- Если вы сжимали один объект, рядом появится файл с тем же именем и расширением **.zip**.
 - Если объектов было несколько, создастся файл с именем «Архив.zip». [1]

Для извлечения файлов: Достаточно дважды кликнуть по **.zip** архиву, и встроенная «Утилита архивирования» автоматически распакует его в ту же папку.

Способ 2: Создание архива с паролем через Терминал

Штатный интерфейс Finder не умеет ставить пароли на ZIP-файлы. Для защиты конфиденциальных данных используется встроенный **Терминал**: [4]

1. Откройте приложение **Терминал** (через поиск Spotlight или в папке *Программы* $\rightarrow$ *Утилиты*).
2. Введите команду **cd** и перетащите из Finder папку, в которой лежат ваши файлы, затем нажмите **Enter** (это переведет Терминал в нужную директорию).
3. Введите команду для создания зашифрованного архива: **zip -er имя_будущего_архива.zip имя_папки_для_сжатия**
4. (Ключ **-e** отвечает за шифрование/пароль, а **-r** — за рекурсивное сжатие всех вложенных файлов).
5. Нажмите **Enter**. Терминал попросит ввести пароль, а затем подтвердить его (при вводе символы на экране отображаться не будут). [4, 5, 6]

Важная особенность Mac-архивов

Когда вы архивируете файлы встроенными средствами macOS, система автоматически добавляет в архив скрытые служебные файлы (например, **.DS_Store** и папку **__MACOSX**). Они нужны для хранения меток, иконок и структуры папок на Mac. [7]

Если этот архив открыть на **Windows**, получатель увидит эти «лишние» мусорные файлы. Чтобы создавать чистые архивы исключительно с вашими данными, лучше использовать Терминал со специальным ключом: [7]

zip -r -X архив.zip папка_с_файлами

(Ключ **-X** исключает сохранение системных метаданных Mac). [7]

Продвинутые программы для macOS

Если вам нужно работать с форматами `.rar` или `.7z` (которые встроенная утилита Mac умеет открывать, но не умеет создавать), используют популярный сторонний софт: [8, 9]

- **Keka** — самый популярный бесплатный архиватор для Mac с открытым кодом. Поддерживает сжатие в 7z, ZIP, TAR и удаляет скрытые файлы macOS одной галочкой.
- **The Unarchiver** — лучшее бесплатное решение для распаковки любых сторонних форматов (особенно старых или созданных на Windows со сложной кодировкой).

Вам требуется архивировать файлы для **передачи пользователям Windows**, или вы хотите автоматизировать этот процесс на Mac?

Алгоритм Хаффмана — это классический метод сжатия данных без потерь, который заменяет символы фиксированной длины (например, 8-битные символы в кодировке ASCII) на коды переменной длины. [1, 2]

Основной принцип прост: **чем чаще символ встречается в тексте, тем короче будет его двоичный код, и наоборот — редкие символы получают длинные коды.** [2]



Главное свойство алгоритма

Код Хаффмана является **префиксным**. Это означает, что ни один код символа не совпадает с началом (префиксом) другого кода. Благодаря этому свойству закодированную последовательность битов можно однозначно расшифровать с начала, не используя разделители между символами. [1, 3, 4]



Как работает алгоритм (пошаговый пример)

Представим, что нам нужно сжать строку: «ABRACADABRA».

Шаг 1. Считаем частоту символов

Сначала алгоритм анализирует текст и считает, сколько раз встречается каждый символ: [2]

- A: 5 раз
- B: 2 раза
- R: 2 раза
- C: 1 раз
- D: 1 раз

Шаг 2. Строим дерево Хаффмана

Мы создаем список «листьев» дерева (каждый символ — это отдельный лист со своим весом-частотой):

[C:1, D:1, B:2, R:2, A:5] [5]

Далее в цикле выполняются следующие действия:

1. Выбираются два узла с минимальным весом. В нашем случае это C:1 и D:1.

2. Они объединяются в один родительский узел, вес которого равен их сумме ($1 + 1 = 2$).
3. Новый узел возвращается в список, а два старых удаляются. [1, 5]

Повторяем процесс, пока не останется одно общее дерево:

- Объединяем C и D $\rightarrow$ узел $(CD) : 2$. Теперь список: [B : 2, R : 2, (CD) : 2, A : 5].
- Объединяем B и R $\rightarrow$ узел $(BR) : 4$. Список: [(CD) : 2, (BR) : 4, A : 5].
- Объединяем $(CD) : 2$ и $(BR) : 4 \rightarrow$ узел $(CDBR) : 6$. Список: [A : 5, (CDBR) : 6].
- Объединяем последние два элемента $\rightarrow$ корень дерева с весом 11 (общая длина строки).

Шаг 3. Присваиваем бинарные коды

Теперь проходим по получившемуся дереву от корня к листьям. Каждому левому повороту (ветви) присваиваем **0**, а каждому правому — **1**. [1, 5]

В результате символы получают следующие уникальные префиксные коды:

- A $\rightarrow$ 0 (всего 1 бит, так как буква самая частая)
- B $\rightarrow$ 110
- R $\rightarrow$ 111
- C $\rightarrow$ 100
- D $\rightarrow$ 101

Шаг 4. Кодировем строку

Заменяем буквы строки ABRACADABRA на новые бинарные коды:

0 110 111 0 100 0 101 0 110 111 0

Обычный текст (11 символов по 8 бит) занял бы **88 бит**.

В сжатом виде строка занимает всего **23 бита**.



Как происходит декодирование

Чтобы прочитать файл, компьютер берет поток битов и идет по дереву Хаффмана сверху вниз:

1. Видит 0 $\rightarrow$ сразу попадает в лист A (записывает букву «А», возвращается наверх).
2. Видит 1 $\rightarrow$ идет вправо, видит 1 $\rightarrow$ вправо, видит 0 $\rightarrow$ влево, попадает в лист B (записывает «В»).
3. Процесс безошибочно повторяется до конца файла. [1, 6]